

ShopAssist — Complete Repository & Backend Guide

Concept-first architecture of the multi-store buyer/seller shopping assistant

Generated from the ShopAssist codebase

July 20, 2026

Contents

1. What this product is	3
2. How to read this document	3
3. Foundational concepts (before the code)	3
3.1 Client–server architecture	3
3.2 REST API	4
3.3 Chat endpoint vs CRUD endpoints	4
3.4 Large Language Model (LLM)	4
3.5 Agent (Google ADK LlmAgent)	4
3.6 Tool (FunctionTool)	5
3.7 Session	5
3.8 Request scope / ContextVar (store_scope)	5
3.9 Store / category vertical	5
3.10 Catalog & knowledge base (KB)	5
3.11 Product tile (<TILES>)	6
3.12 Inventory status & visibility	6
3.13 Cart, reservation, order	6
3.14 Shop request / store query	7
3.15 CRM (Customer Relationship Management)	7
3.16 Vision & pending chat image	7
3.17 Deterministic commerce handlers (ShopAssist mode)	7
3.18 Speech (STT / TTS)	7
4. Repository layout (big picture)	8
5. How the backend starts	8
6. Request lifecycle: POST /ask (the heart of the system)	9
7. API surface (what connects to what)	9
7.1 Defined on app.py	9
7.2 api/routes/core.py	10
7.3 api/routes/stores.py	10

7.4 api/routes/seller.py	10
7.5 api/routes/cart.py	10
7.6 api/routes/seller_ai.py (prefix /seller/ai)	10
8. Agents package (deep dive)	11
8.1 Buyer agent (agents/static_agent.py)	11
8.2 Seller agent (agents/seller_agent.py)	11
8.3 ShopAssist agents	11
9. Tools package (deep dive)	12
9.1 search_kb (buyer grounding)	12
9.2 CRM tools	12
9.3 Seller inventory tools	12
9.4 Similarity from photo	12
10. Catalog package	12
11. Commerce package	13
12. Stores, persistence, media, voice, config	14
12.1 Stores	14
12.2 Persistence	14
12.3 Media	14
12.4 Voice	14
12.5 Config	14
13. Data files (runtime state)	14
14. Dual write paths: agent tools vs REST	15
15. End-to-end flows (backend)	15
15.1 Buyer browses and gets cards	15
15.2 Buyer asks for missing item	15
15.3 Seller lists from photo	15
15.4 Cart checkout	16
15.5 Seller morning brief	16
16. Frontend overview (botbotbot/)	16
17. Connection map (mental model)	16
18. Modes compared	17
19. Operational checklist	17
20. Glossary (quick reference)	18

1. What this product is

ShopAssist (also referred to in code as an enterprise CRM / multi-store boutique assistant) is a proof-of-concept shopping platform with two human roles:

- **Buyer** — browses a store by category (Skincare, Apparel, Handicrafts), chats with an AI store assistant, sees product cards, can add to cart, checkout, and ask for items the store does not have.
- **Seller** — owns one or more shops, manages inventory via chat or forms, answers buyer questions, and uses seller-AI helpers (morning brief, pricing, restock priorities, etc.).

The system is split into:

Layer	Folder	Technology
Backend API + AI agents	adk/	Python, FastAPI, Google ADK, Postgres, JSON files
Mobile client	botbotbot/	Expo / React Native (Expo Router)
Infrastructure	docker-compose.yml	Postgres (pgvector image) for sessions & CRM

Default mode is **CRM / multi-store** (`AGENT_MODE=crm`). An alternate **ShopAssist marketplace** mode (`AGENT_MODE=shopassist`) reuses much of the same HTTP surface but swaps agents and adds deterministic commerce shortcuts.

2. How to read this document

This guide explains **concepts first**, then **how this repository uses them**. That order is intentional: if you meet a term (Agent, Tool, Session, Tile, Store scope), you already know what the idea means before seeing file names and call graphs.

Sections 3–6 are foundation concepts. Sections 7–15 map those concepts onto the backend packages. Section 16 covers the mobile app briefly. Section 17 is an end-to-end connection map.

3. Foundational concepts (before the code)

3.1 Client–server architecture

Concept. A *client* (phone app) never talks to the LLM or database directly. It sends HTTP requests to a *server* (backend). The server owns business rules, secrets (API keys), and data.

Why it helps. Security, one source of truth for inventory/cart, and a single place to swap LLM providers.

In this repo. The Expo app (botbotbot) calls FastAPI on port 8000 (adk). Base URL comes from `EXP0_PUBLIC_API_URL`.

3.2 REST API

Concept. REST exposes *resources* via HTTP methods: GET to read, POST to create/act, DELETE to remove. Bodies are usually JSON. Paths look like `/stores`, `/cart/add`, `/seller/products`.

Why it helps. Predictable contracts for inventory, stores, cart, and notifications — work that should not depend on an LLM guessing tool calls.

In this repo. Route modules under `adk/api/routes/` plus chat endpoints on `adk/api/app.py`.

3.3 Chat endpoint vs CRUD endpoints

Concept. Not every user action is “ask the AI.”

- **Chat endpoint** (POST `/ask`): natural language -> agent -> tools -> answer (+ optional product tiles).
- **CRUD / utility endpoints**: explicit forms and buttons (create store, patch price, checkout).

Why it helps. Forms stay reliable; chat stays flexible. Both share the same catalogs so chat and UI stay in sync.

3.4 Large Language Model (LLM)

Concept. An LLM is a model that predicts text. Given instructions + conversation + tool results, it produces a reply (and may request tool calls).

Why it helps. Buyers can ask “something for dry skin”; sellers can say “add 10 units of the blue shirt” without rigid menus for every phrase.

In this repo. Configured in `adk/config/llm_config.py` via `LLM_PROVIDER` (openai default via LiteLLM, or gemini). Keys live in `adk/.env`.

3.5 Agent (Google ADK LlmAgent)

Concept. An *agent* wraps an LLM with:

1. A system **instruction** (role, rules, tone).
2. A set of **tools** the model may call.
3. A **runner** that executes the loop: think -> call tools -> observe -> answer.
4. A **session service** so multi-turn chat has continuity.

Why it helps. Separates “who the assistant is” (buyer support vs seller ops) from “how HTTP works.”

In this repo.

Agent	File	Role
<code>boutique_support_agent</code>	<code>agents/static_agent.py</code>	Buyer / store support
<code>seller_ops_agent</code>	<code>agents/seller_agent.py</code>	Seller inventory ops
ShopAssist agents	<code>agents/shopassist_*.py</code>	Marketplace mode

3.6 Tool (FunctionTool)

Concept. A *tool* is an ordinary Python async function whose **docstring and signature** become the schema the LLM sees. When the model needs facts or mutations, it calls the tool; the backend runs real code and returns a string (often with a <TILES> block).

Why it helps. The model does not invent stock or write files itself — tools enforce truth and side effects.

In this repo. `adk/tools/agent_tools.py` (buyer CRM + KB search) and `adk/tools/seller_agent_tools.py` (inventory CRUD, similarity, analytics helpers).

3.7 Session

Concept. A *session* is a conversation thread keyed by `session_id`. It stores ADK agent state and (separately) human-readable chat history.

Why it helps. “Change the price of that one” only works if the system remembers prior turns and pending photos.

In this repo.

- ADK: `DatabaseSessionService` (CRM) or in-memory (ShopAssist) against `Postgres DATABASE_URL`.
- App history: `persistence/session_store.py` (`SessionHistory`, `append_message`, `load_messages`).

3.8 Request scope / ContextVar (store_scope)

Concept. HTTP is stateless, but one request may fan out into many tool calls. *Request scope* stores `store_id`, `role`, `session_id` in a context variable (and fallback map) for the duration of that request so tools can read “which shop am I in?” without every tool argument being perfect.

Why it helps. Prevents cross-store leaks and simplifies tool signatures.

In this repo. `adk/stores/store_scope.py`; set at the start of `/ask`.

3.9 Store / category vertical

Concept. A *store* is a shop record: `id`, `name`, `owner metadata`, and a category vertical (Skincare / Apparel / Handicrafts). Buyers shop inside one store at a time; sellers operate on their store’s inventory.

Why it helps. Multi-tenant demo without a full multi-tenant DB schema — scoping is by `store_id` on products and queries.

In this repo. `data/stores.json` + `stores/store_registry.py` + `/stores` routes.

3.10 Catalog & knowledge base (KB)

Concept.

- A **catalog** is structured product records (`SKU`, `price`, `qty`, `tags`, `images`, `status`).

- A **knowledge base** is prose (or chunks) the agent searches for answers (specs, care, ingredients).

Why it helps. Buyers get grounded answers; sellers mutate a live catalog that can also feed search.

In this repo.

- Seed KB: `data/fake_kb.md` searched by `search_kb`.
- Live inventory: `data/seller_products.json` via `catalog/seller_catalog.py`.
- ShopAssist marketplace catalog modules under `catalog/shopassist_*`.

3.11 Product tile (<TILES>)

Concept. Agents must not dump raw markdown image galleries. Instead they emit a tagged JSON array:

Short sentence.

```
<TILES>[{"id":"SK-...", "name":"...", "price":"$28", "img":"...", ...}]</TILES>
```

The mobile app parses that markup and renders tappable `ProductTileCards`.

Why it helps. Stable UI contracts: chat remains conversational; cards remain structured.

In this repo. Produced by tools/commerce helpers; sanitized in `commerce/tile_validator.py`; parsed in `botbotbot/src/shared/utils/parseTiles.ts`.

3.12 Inventory status & visibility

Concept. Each SKU has a lifecycle status: **active** (buyers can see), **draft** (seller-only), **trash** (soft-deleted). A separate visibility map can sync what the app considers authoritative.

Why it helps. Sellers can stage listings; buyers only see published stock.

In this repo. Fields on seller products + `data/inventory_visibility.json` + `POST /shop/inventory-visibility`.

3.13 Cart, reservation, order

Concept.

- **Cart** — pending lines for a buyer before payment.
- **Reservation** — holding stock quantity when an item is added so two buyers do not oversell the same unit in a demo.
- **Order** — snapshot after checkout.

Why it helps. Turns chat discovery into commerce without a full payment gateway.

In this repo. `commerce/buyer_cart.py` + `data/buyer_carts.json` / `buyer_orders.json` + `/cart/*` routes. ShopAssist mode also has session-scoped commerce in Postgres session docs.

3.14 Shop request / store query

Concept. When a buyer wants something not in catalog, the system *logs* the ask for the owner instead of inventing a product. That log appears as a store query / shop request.

Why it helps. Captures demand; seller can answer later (optionally with AI draft).

In this repo. Tool `log_shop_request`, CRM tables, `data/store_queries/store_*.json`, routes under `/stores/{id}/queries` and `/shop/requests`.

3.15 CRM (Customer Relationship Management)

Concept. Lightweight CRM tracks a contact per chat session: status (new -> interested -> order_pending -> ...), notes, follow-ups, escalations, shop requests.

Why it helps. The buyer agent can “remember” relationship stage and escalate angry customers without a separate CRM product.

In this repo. `persistence/crm_models.py` (SQLAlchemy + Postgres).

3.16 Vision & pending chat image

Concept. Sending huge base64 images into every LLM turn is expensive and noisy. Instead the API **stashes** the upload on disk keyed by `session_id`. Later tools say `use_pending_chat_image=true`. Vision models can also *guess* product fields or find *similar* inventory.

Why it helps. Chat-first listing and “do I already sell something like this?” without stuffing binary into prompts.

In this repo. `media/chat_image_stash.py`, `static/pending_chat_images/`, `catalog/product_vision_guess.py`, `catalog/inventory_similarity.py`.

3.17 Deterministic commerce handlers (ShopAssist mode)

Concept. Some intents (“add to cart”, “compare A and B”, “show cheapest under \$30”) are better handled by **rules and catalog filters** than by free-form LLM output. Handlers run *before* the agent and may short-circuit with a complete answer.

Why it helps. Accuracy and speed for cart/browse/compare; LLM used when language is open-ended.

In this repo. `commerce/session_commerce.py`, `intent_router.py`, `browse_filters.py`, `comparison_handlers.py`, etc., wired in `api/app.py` when `AGENT_MODE=shopassist`.

3.18 Speech (STT / TTS)

Concept. **STT** (speech-to-text) turns audio into a query string; **TTS** (text-to-speech) turns the answer into audio. Same chat pipeline underneath.

Why it helps. Hands-free shopping / seller ops demos.

In this repo. `voice/whisper_utils.py` (OpenAI Whisper) -> `/stt`, `/ask_voice`.

4. Repository layout (big picture)

```
final_chatbot-main 2/
├── README.md           # Quick start
├── docker-compose.yml  # Postgres for ADK sessions + CRM
├── adk/                # Backend (this guide's focus)
└── botbotbot/          # Expo mobile client
```

Inside adk/ the important packages are:

Package	Responsibility
api/	FastAPI app, Pydantic schemas, HTTP routes
agents/	LlmAgent + Runner definitions
tools/	Functions exposed to agents
catalog/	Products, tagging, vision, similarity
commerce/	Tiles, cart, browse, seller AI, ShopAssist handlers
stores/	Store registry + request scope
persistence/	Postgres sessions + CRM models
config/	LLM provider wiring
media/	Pending chat image stash
voice/	Whisper STT/TTS
data/	Runtime JSON + fake_kb.md
static/	Served product & pending images
scripts/	Seed / image download utilities
Root *.py shims	Re-export package modules for older import paths
main.py	uvicorn main:app entry
paths.py	Central path constants

Many root-level .py files in adk/ are **thin shims** (from `catalog.seller_catalog import *` style) so older imports keep working after the package split.

5. How the backend starts

1. Docker Compose starts Postgres (DATABASE_URL, often port 5433).
2. `cd adk && uvicorn main:app --host 0.0.0.0 --port 8000 --reload`
3. `main.py` imports app from `api.app`.
4. FastAPI **lifespan** initializes CRM schema and may seed demo stores.
5. On import, `AGENT_MODE` selects which agent runners are loaded.
6. Routers are mounted; static files mount at `/product-images`.
7. Client hits `GET /health` to verify.

Environment (from `.env.example`):

Variable	Purpose
LLM_PROVIDER / OPENAI_* / GOOGLE_*	Which model stack
AGENT_MODE	crm (default) or shopassist
AGENT_USE_TOOLS	ShopAssist: tools agent vs static catalog-in-prompt
API_PUBLIC_URL	Absolute URLs embedded in image links
DATABASE_URL	Postgres for ADK + CRM

Dependencies (requirements.txt): FastAPI, uvicorn, pydantic, google-adk, google-genai, openai, litellm, asyncpg, sqlalchemy, psycopg, pymongo, python-multipart, dotenv.

6. Request lifecycle: POST /ask (the heart of the system)

Concept first. A chat turn is: authenticate nothing (demo) -> identify role & store -> optionally stash image -> optionally run deterministic handlers -> run the correct agent runner -> sanitize markup -> persist history -> return { answer, ... }.

How it works here (api/app.py):

1. Client sends query, session_id, role (buyer/seller), optional store_id / store tag, optional image.
2. set_request_scope(...) binds store/role/session for tools.
3. If image present -> save under pending_chat_images/ for that session.
4. If ShopAssist mode -> try commerce / browse / compare / best-pick short-circuits.
5. Inject SYSTEM NOTE hints (session id, store tag, active product, anti-repeat lists).
6. Choose **buyer runner** or **seller runner**.
7. ADK Runner executes the agent loop (LLM <-> tools).
8. Response text is normalized/sanitized (tile_validator); history appended in Postgres.
9. JSON response returned to the app; app parses TILES into cards.

Voice path: /ask_voice runs STT -> same pipeline -> optional TTS audio stream.

7. API surface (what connects to what)

7.1 Defined on app.py

Endpoint	Job
POST /ask	Main chat
POST /ask_voice	Voice chat
POST /stt	Transcription only
Static /product-images	Product photos

7.2 api/routes/core.py

Endpoint	Job
GET /health	Liveness
GET /shop/requests	List unmet buyer asks (CRM)
POST /shop/requests/{id}/fulfill	Mark fulfilled
GET /products/{product_id}	Product detail
POST /session/active-product	Focus product for ShopAssist turns
GET /session/{id}/history	Chat history

7.3 api/routes/stores.py

Endpoint	Job
GET/POST /stores	List / create shops
GET /stores/{id}	One shop
GET /stores/{id}/queries	Buyer questions for owner
POST .../queries/{qid}/answer	Owner reply

7.4 api/routes/seller.py

Endpoint	Job
GET/POST /seller/products	List / upsert inventory
POST /seller/products/retag	Retag products
DELETE /seller/products/{sku}	Remove
POST /seller/product-from-image	Vision field guess
POST /shop/inventory-visibility	Sync active/draft/trash map

7.5 api/routes/cart.py

Endpoint	Job
GET /cart, POST /cart/add update remove clear	Cart CRUD
POST /cart/checkout, POST /buy-now	Orders
POST /notify/subscribe	Restock waitlist
GET /notify/count, POST /notify/broadcast, GET /notify/inbox	Seller broadcast -> buyer inbox

7.6 api/routes/seller_ai.py (prefix /seller/ai)

Morning brief, query draft, listing-from-image, restock notify/priorities, pricing suggestion, promo copy, buyer intent, store analytics, batch photos, translate reply — implemented in commerce/seller_ai.py.

Shared contracts live in `api/schemas.py` (Pydantic models for request/response bodies).

Shared mode flags / runners live in `api/deps.py`.

8. Agents package (deep dive)

8.1 Buyer agent (`agents/static_agent.py`)

Instruction concept. Act as frontline boutique support for *one* store category; never invent products; always search KB first.

Tools:

Tool	Concept	Function
<code>search_kb</code>	Retrieval over KB + live overlay	Ground product answers; emit TILES
<code>get_contact</code>	Read CRM contact	Know buyer stage
<code>update_contact</code>	Write CRM status	Pipeline hygiene
<code>create_followup</code>	Note for owner	Deferred human work
<code>log_shop_request</code>	Demand capture	Missing-item logging
<code>escalate_to_human</code>	Safety valve	Angry / refund / “talk to owner”

Uses `DatabaseSessionService`, app name `enterprise_crm_app`.

8.2 Seller agent (`agents/seller_agent.py`)

Instruction concept. Lazy-seller UX: do inventory work *in chat*; prefer cards; use pending photo; never redirect to menus.

Tools: list inventory, low stock, find similar from photo, restock priorities, pricing suggestion, analyze buyer questions, store analytics, upsert / update field / remove / permanent delete — plus limited CRM/KB tools.

Uses separate app name `seller_ops_app` so seller sessions do not collide with buyer sessions.

8.3 ShopAssist agents

- **Tool agent** (`shopassist_agent.py`): marketplace search tools.
- **Static agent** (`shopassist_static_agent.py`): catalog baked into the prompt (`AGENT_USE_TOOLS=false`).

Prompts live under `agents/prompts/`.

9. Tools package (deep dive)

9.1 search_kb (buyer grounding)

Concept. *Retrieval* scores text chunks against a query (with intent expansion: “serum” -> skincare terms). Results include facts and TILES from matching products.

How it helps functioning. Enforces “only recommend what we stock / know,” scoped to the current store category, and merges seed KB with live seller products where applicable.

9.2 CRM tools

Concept. Session id -> contact row. Statuses like new, interested, order_pending, requires_human, resolved.

How it helps. Buyer agent maintains a lightweight funnel without a separate CRM UI.

9.3 Seller inventory tools

Concept. Chat is a CRUD front-end over seller_catalog with store scoping and SKU generation (SK-NEW-..., AP-NEW-..., HC-NEW-...).

How it helps. Sellers manage stock conversationally; TILES from list tools feed the same card UI as REST.

9.4 Similarity from photo

Concept. Compare vision-derived type/features against inventory embeddings/heuristics (inventory_similarity.py).

How it helps. “Do I already have something like this photo?” without forcing the model to list-and-guess by name.

10. Catalog package

Module	Concept	Role in system
seller_catalog.py	Live inventory store	Read/write seller_products.json, visibility sync
boutique_catalog.py	Boutique helpers	Boutique-oriented product views
shopassist_catalog.py	Marketplace catalog	ShopAssist mode product universe
/shopas-		
sist_data.py /		
shopas-		
sist_pid_map.py		
product_tagger.py	Tagging	Auto tags for search/filter

Module	Concept	Role in system
product_images.py / prod- uct_image_pools.py	Image maps & pools	Attach photos to SKUs
product_vision_guess.py inventory_similarity.py	Multimodal field guess Similarity search	Prefill listing from photo Photo -> similar SKUs

All paths resolve through paths.py so scripts and agents agree on file locations.

11. Commerce package

Think of commerce as **everything that shapes shopping answers without being “the agent definition.”**

Module	Concept	Function
tile_validator.py	Markup contract	Sanitize TILES/tables; fast browse responses
product_matcher.py	Resolve product ids/names	Detail lookups, “already discussed” lists
session_commerce.py buyer_cart.py	Session cart / active product Persistent buyer cart	ShopAssist commerce state Cross-shop cart JSON + reserve stock
intent_router.py	Intent classification	Route to handlers
browse_filters.py	Faceted browse	Price/category filters in session
comparison_handlers.py	Compare N products	Deterministic compare answers
combo_handlers.py	Show+describe combos	Multi-step visual replies
best_recommendation.py	“Best for X”	Heuristic best-pick
prior_items.py	Refer to earlier items	Continuity
conversation_context.py	Pronoun / “that one”	Resolve references
upsell_policy.py	Soft upsell rules	Limit pushy suggestions
visual_handoff.py	Text before visuals	Never silent tile dumps
boutique_response.py	Boutique formatting	CRM-mode response shaping
query_understand.py	Query parse helpers	Shared NLU-ish utilities
session_reset.py	Clear session commerce	Fresh start
seller_ai.py	Seller copilots	Briefs, drafts, pricing, analytics APIs

12. Stores, persistence, media, voice, config

12.1 Stores

- `store_registry.py` — load/save `stores.json`, normalize categories, seed demos.
- `store_scope.py` — per-request `ContextVar` for tools.

12.2 Persistence

- `session_store.py` — chat message history APIs used by `/ask` and `/session/.../history`.
- `crm_models.py` — SQLAlchemy models: contacts, notes, interactions, shop requests; async engine from `DATABASE_URL`.

Concept. Separate *agent session state* (ADK) from *displayable transcript* (`SessionHistory`) and from *CRM entities* — three memories with different jobs.

12.3 Media

`chat_image_stash.py` writes pending images (and optional `.vision.json` sidecars) under `static/pending_chat_images/`.

12.4 Voice

`whisper_utils.py` wraps OpenAI audio endpoints; gated by `is_openai_configured()`.

12.5 Config

`llm_config.py` builds ADK agent kwargs for OpenAI-via-LiteLLM or Gemini based on `LLM_PROVIDER`.

13. Data files (runtime state)

File / directory	Stores
<code>data/stores.json</code>	Shop registry
<code>data/seller_products.json</code>	Inventory by SKU
<code>data/inventory_visibility.json</code>	Active/Draft/Trash authority map
<code>data/store_queries/*.json</code>	Per-store buyer Q&A threads
<code>data/fake_kb.md</code>	Seed knowledge base
<code>data/buyer_carts.json</code>	Carts by <code>buyer_id</code>
<code>data/buyer_orders.json</code>	Checkout history
<code>data/restock_notify.json</code>	Waitlist by SKU
<code>data/product_images.json</code> / <code>boutique_product_images.json</code>	Image URL maps
<code>static/products/</code>	Files served as <code>/product-images/...</code>
<code>static/pending_chat_images/</code>	Ephemeral chat uploads

File / directory	Stores
Postgres	ADK sessions + CRM + message history

JSON is used for demo speed and inspectability; Postgres for durability of conversations and CRM.

14. Dual write paths: agent tools vs REST

Concept. The same inventory can change via:

1. Seller chat -> upsert_inventory_item tool -> seller_catalog.
2. Seller form -> POST /seller/products -> seller_catalog.

Why it helps. Chat-first *and* form-first UX without divergent databases.

Same pattern for queries (tool log_shop_request vs safety nets writing store_queries) and for seller AI (tools inside chat vs /seller/ai/* buttons).

15. End-to-end flows (backend)

15.1 Buyer browses and gets cards

1. App: role buyer -> category -> store -> chat.
2. POST /ask with role=buyer, store_id.
3. Buyer agent calls search_kb.
4. Tool returns facts + <TILES>.
5. Agent replies with one sentence + exact TILES.
6. App parses tiles -> product screen via /products/{id} or tile URL.

15.2 Buyer asks for missing item

1. search_kb no match.
2. Agent calls log_shop_request.
3. CRM +/- store_queries updated.
4. Seller Queries screen loads via /stores/{id}/queries.
5. Optional POST /seller/ai/query-draft for a reply draft.

15.3 Seller lists from photo

1. App uploads image with /ask or vision endpoint.
2. Image stashed; vision may guess fields.
3. Seller says "list this" -> upsert_inventory_item(use_pending_chat_image=true).
4. Or "do I have this?" -> find_similar_inventory_from_photo.
5. Inventory JSON + image under static/products/ updated; cards shown.

15.4 Cart checkout

1. App uses persistent buyer_id.
2. POST /cart/add reserves qty in seller_products / cart JSON.
3. POST /cart/checkout writes buyer_orders.json.
4. Out-of-stock path may POST /notify/subscribe; seller broadcast fills buyer inbox.

15.5 Seller morning brief

1. App FAB calls GET /seller/ai/morning-brief?store_id=...
2. commerce/seller_ai.py aggregates low stock, open queries, priorities.
3. Returns structured brief for SellerMorningBrief UI.

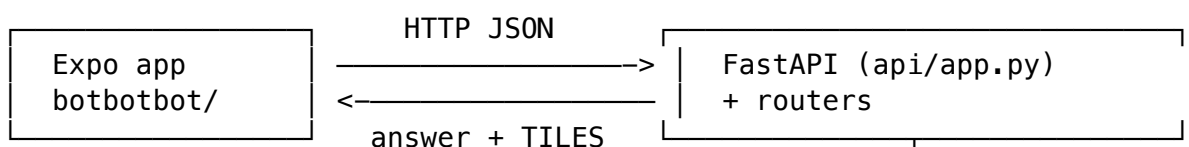
16. Frontend overview (botbotbot/)

Concept. Expo Router maps URLs to screens; feature folders own UI; src/services/* own HTTP.

Area	Role
features/role-picker	Choose buyer vs seller
features/buyer	Shop chat, notify FAB
features/seller	Seller chat, queries, AI brief, add-product cards
features/cart	Cart UI
features/inventory	Inventory management
features/product	Product detail
features/chat-shared	Shared chat chrome / tiles
services/api-fetch.ts	/ask, /ask_voice
services/storesApi.ts	Stores & seller products
services/cartApi.ts	Cart & checkout
services/sellerAiApi.ts	/seller/ai/*
services/notifyApi.ts / buyerNotifyApi.ts	Broadcast & inbox
services/visionGuessApi.ts	Product-from-image
shared/utis/parseTiles.ts	TILES -> card models

The frontend is a **consumer** of the concepts above; it does not run agents locally.

17. Connection map (mental model)



7. Smoke: seller create store -> list item; buyer chat -> tiles; missing item -> queries; cart checkout
-

20. Glossary (quick reference)

Term	Meaning
ADK	Google Agent Development Kit — agents, runners, tools, sessions
Agent	LLM + instructions + tools
Tool	Backend function the LLM may call
Session	Conversation continuity id
Store scope	Request-local shop/role context
TILES	JSON product cards embedded in chat text
SKU	Stock-keeping unit / product id
KB	Knowledge base (fake_kb.md + search)
CRM	Contact/notes/escalation tables in Postgres
Visibility	Active/Draft/Trash publication map
Pending image	Stashed chat photo for later tools
Runner	Executes the agent tool loop
Shim	Root .py re-export for old imports

End of guide. Generated to document the ShopAssist repository structure and backend concepts as implemented in the codebase.